

Logic Building Tasks in Python

BEGINNER LEVEL – Logic & Fundamentals

Focus: Problem decomposition, loops, conditionals, arrays, and string logic. DSA focus: Lists, dictionaries, sets, tuples.

- 1 Reverse a number without using string conversion.
- 2 Find the largest/smallest of three numbers.
- 3 Check if a number is prime.
- 4 Find all factors of a number.
- 5 Compute GCD and LCM (Euclidean algorithm).
- 6 Generate Fibonacci sequence up to n.
- 7 Check if a string is a palindrome (without slicing).
- 8 Count vowels, consonants, digits, and spaces in a string.
- 9 Find the second largest element in a list without sorting.
- 10 Sum of all even and odd numbers in a list separately.
- 11 Find duplicates in a list using sets.
- 12 Remove all occurrences of a specific element.
- 13 Merge two sorted lists into one sorted list (without built-in functions).
- 14 Frequency count of each element (dictionary logic).
- 15 Rotate list elements left/right by k positions.
- 16 Sort list using bubble/selection/insertion sort (manual implementation).
- 17 Find missing number in a sequence (sum formula).
- 18 Check if two strings are anagrams.
- 19 Find intersection and union of two lists using sets.
- 20 Pattern printing (pyramids, diamonds, numbers).

INTERMEDIATE LEVEL – Structured Problem Solving

Focus: Applying core DSA (arrays, stacks, queues, hashmaps, recursion). DSA focus: Arrays, Stacks, Queues, Linked Lists, Hashmaps.

- 1 Implement a Stack (list-based and class-based).
- 2 Implement a Queue (list-based and class-based).
- 3 Evaluate a postfix expression using Stack.
- 4 Balanced parentheses check using Stack.
- 5 Next Greater Element problem.
- 6 Stock Span problem.
- 7 Sliding Window Maximum / Minimum (Deque logic).
- 8 Subarray with sum equal to k (Prefix Sum + Hashmap).

- 9 Longest substring without repeating characters.
- 10 Move all zeros to the end of the array (in-place).
- 11 Kadane's algorithm (maximum subarray sum).
- 12 Find first non-repeating character in a string.
- 13 Detect loop in linked list (Floyd's cycle detection).
- 14 Reverse a linked list iteratively and recursively.
- 15 Merge two sorted linked lists.
- 16 Find the middle element of a linked list.
- 17 Binary search (iterative and recursive).
- 18 Implement sorting algorithms: Merge sort, Quick sort.
- 19 Find pair with a given sum in a sorted array.
- 20 Generate all subsets of a given list (recursion / bit manipulation).

ADVANCED LEVEL – Problem Solving & Algorithmic Design

Focus: Dynamic Programming, Graphs, Trees, Backtracking, Greedy, and advanced logic patterns.
DSA focus: Trees, Graphs, Heaps, DP, Tries.

- 1 Binary Tree traversals (inorder, preorder, postorder).
- 2 Level Order Traversal (BFS).
- 3 Check if two trees are identical.
- 4 Find Lowest Common Ancestor (LCA) in a Binary Tree.
- 5 Diameter of a Binary Tree.
- 6 Convert a Binary Tree to its mirror.
- 7 Depth of Binary Tree (recursive + iterative).
- 8 Check if Binary Tree is Balanced.
- 9 Dijkstra's algorithm (shortest path).
- 10 Topological sort (DFS and Kahn's algorithm).
- 11 Detect cycle in directed and undirected graph.
- 12 Flood Fill algorithm (BFS/DFS grid problem).
- 13 Number of islands in a grid (DFS).
- 14 Knapsack problem (0/1 and fractional).
- 15 Coin Change (minimum coins / total combinations).
- 16 Longest Increasing Subsequence (DP).
- 17 Matrix chain multiplication (DP).
- 18 Word Break problem (DP + Hashset).
- 19 Sudoku solver (backtracking).
- 20 N-Queens problem (backtracking).
- 21 Trie data structure (insert, search, prefix).
- 22 Huffman Coding implementation (Greedy + Heap).

- 23 LRU Cache (OrderedDict / LinkedHashMap logic).
- 24 Minimum Spanning Tree (Kruskal / Prim).
- 25 A* Search Algorithm (AI heuristic pathfinding).